

Functional Requirements

Document Information

Project: Blogging System API

Version: 1.0

Introduction

This document defines the functional requirements of the Blogging System API. Each requirement describes a capability that the system must provide. Every implemented feature should be traceable to at least one functional requirement listed in this document.

Functional Requirement Categories

- Authentication
 - Authorization
 - User Management
 - Blog Management
 - Comment Management
 - Security
 - Data Persistence
 - Logging
 - API Behaviour
-

Authentication Requirements

FR-001 User Registration

The system shall allow any new user to register using:

- First Name
- Surname
- Email Address
- Password

The system shall reject duplicate email addresses.

FR-002 Password Validation

The system shall validate passwords before account creation.

Passwords must:

- Be at least eight (8) characters long.
 - Contain at least one uppercase letter.
 - Contain at least one lowercase letter.
 - Contain at least one numeric digit.
 - Contain at least one special character.
-

FR-003 Password Encryption

The system shall hash every user password using BCrypt before storing it in the database.

Plain text passwords shall never be stored.

FR-004 Email Verification

After successful registration:

- A six-digit OTP shall be generated.
 - The OTP shall be sent to the user's registered email using SMTP.
 - The account shall remain unverified until the correct OTP is provided.
-

FR-005 Login

The system shall authenticate users using:

- Email
- Password

Only verified accounts shall be allowed to log in.

FR-006 Access Token

Upon successful authentication, the system shall generate a JWT Access Token.

FR-007 Refresh Token

Upon successful authentication, the system shall generate a Refresh Token for session renewal.

FR-008 Forgot Password

Users shall be able to request a password reset using their registered email address.

FR-009 Password Reset OTP

When a password reset is requested:

- A six-digit OTP shall be generated.
 - The OTP shall be emailed.
 - The OTP shall expire after a configurable period.
-

FR-010 Reset Password

Users shall be able to reset their password after successful OTP verification.

FR-011 Change Password

Authenticated users shall be able to change their password by providing:

- Current Password
 - New Password
-

Authorization Requirements

FR-012 User Roles

The system shall support two roles:

- Administrator
 - User
-

FR-013 Role Permissions

The system shall restrict API endpoints based on user roles.

Unauthorized users shall receive an appropriate HTTP response.

User Management Requirements

FR-014 User Profile

Each user shall possess:

- First Name
 - Surname
 - Email
 - Avatar (optional)
 - Bio (optional)
-

FR-015 Unique Email

Each email address shall be unique across the system.

FR-016 Verified Accounts

Only verified users shall access protected endpoints.

Blog Management Requirements

FR-017 Create Post

Administrators shall be able to create blog posts.

FR-018 Edit Post

Administrators shall be able to update their own posts.

FR-019 Delete Post

Administrators shall permanently delete their own posts.

FR-020 Draft Posts

Administrators shall be able to save blog posts as drafts.

FR-021 Publish Draft

Administrators shall be able to publish draft posts.

FR-022 View Published Posts

Users shall only be able to view published posts.

Draft posts shall not be visible to users.

FR-023 Markdown Support

The system shall store blog content as Markdown.

FR-024 Post Summary

Each blog post shall contain a summary.

FR-025 Comment Locking

Administrators shall be able to enable or disable comments on their own posts.

When comments are locked:

- New comments shall not be accepted.
 - Existing comments shall not be returned by the API.
 - Clients shall display no comments for the locked post.
 - If comments are later unlocked, all previously stored comments shall become visible again.
-

Comment Requirements

FR-026 Add Comment

Authenticated users shall be able to comment on published blog posts.

FR-027 Reply to Comment

Authenticated users shall be able to reply to comments.

FR-028 Unlimited Nesting

Replies shall support unlimited nesting through recursive parent-child relationships.

FR-029 Administrator Replies

Administrators shall be able to reply to comments on blog posts they own.

FR-030 Comment Visibility

All comments shall remain publicly visible unless:

- The administrator has locked comments for the post.
 - The comment has been permanently deleted.
-

FR-031 Delete Comment

Users shall be able to permanently delete only comments they authored.

Administrators shall be permitted to permanently delete any comment when necessary for moderation purposes.

Deleted comments shall be permanently removed from the database.

Data Persistence Requirements

FR-032 PostgreSQL Storage

Application data shall be stored in PostgreSQL.

FR-033 Entity Framework Core

Entity Framework Core shall manage database access.

FR-034 Database Integrity

Foreign key constraints shall enforce relationships between entities.

Logging Requirements

FR-035 OpenTelemetry

The application shall implement OpenTelemetry for application observability.

FR-036 Console Export

Telemetry data shall be exported to the application console.

API Requirements

FR-037 RESTful Endpoints

The application shall expose RESTful HTTP endpoints.

FR-038 JSON Responses

All API responses shall use JSON.

FR-039 Standard Response Wrapper

Every API response shall follow the standard response format.

Example:

```
{
  "success": true,
  "message": "Operation completed successfully.",
  "data": {}
}
```

FR-040 HTTP Status Codes

The application shall return appropriate HTTP status codes for all operations.

Examples include:

- 200 OK
 - 201 Created
 - 204 No Content
 - 400 Bad Request
 - 401 Unauthorized
 - 403 Forbidden
 - 404 Not Found
 - 409 Conflict
 - 500 Internal Server Error
-

FR-041 API Documentation

All endpoints shall be documented using Swagger/OpenAPI.

FR-042 API Testing

All implemented endpoints shall be verified using a Postman collection.

Functional Requirement Traceability

Every implementation task must reference one or more Functional Requirement IDs (FR-001 through FR-042).

No feature shall be implemented unless it satisfies a documented functional requirement.